

5th Meeting 11/25

Literature Papers that talk similar to my method (WorkerNet)

1. YOLOP

[2108.11250_compressed.pdf](#)

This paper solves detection + segmentation + lane detection in ONE network instead of three separate models → exactly what WorkerNet does but for detection + pose + barcode. Shows unified architecture works better than sequential models.

2. GradNorm

[1803.10704v2.pdf](#)

It automatically balances gradients between three tasks (detection, pose, barcode) during training so one task doesn't dominate and crash the whole thing.

3. ED-Pose

[577_explicit_box_detection_unifies-compressed.pdf](#)

Unifies detection and pose in single forward pass without heatmaps → the EXACT problem WorkerNet addresses. Shows how global (detection) and local (pose) features work together better than separately.

4. HybridPose

[2305.01167_compressed.pdf](#)

Single-stage network predicting detection boxes + keypoints + visibility maps simultaneously, matching detection-pose-barcode design. Shows hybrid approach beats pure regression or heatmap methods alone.

5. Generalist YOLO

[Chang_Generalist_YOLO_Towards_Real-Time_End-to-End_Multi-Task_Visual_Language_Models_WACV_2025_paper.pdf](#)

Handles 5+ vision+language tasks in ONE model at 135x faster than separate models. Proves unified architecture with proper encoder design destroys task interference → directly applicable to this architecture design.

6. Federated Worker Action Recognition

[Federated Action Recognition for Smart Worker Assistance Using FastPose - 2508.14113v1.pdf](#)

Uses FastPose for worker pose extraction then recognizes worker gestures (grab, stop, left, right, up, down) → literally bottle quality checking workflow but with privacy preservation via federated learning.

7. MDSP Vehicle Occupancy

[2205.01515v1.pdf](#)

First unified network combining detection + segmentation + pose for vehicle monitoring = the exact three-task design but for cars instead of bottles. Proves

joint training improves accuracy over single-task.

8. YOLO-Pose

[2204.06806v1.pdf](#)

Joint detection + 2D pose in single YOLO forward pass without heatmaps using OKS loss. Achieves 90.2% AP50 on COCO → direct blueprint for the detection-pose integration with unified loss.

9. Machine Learning + Blockchain Worker Monitoring

[Using machine learning and blockchain for trusted detection and monitoring of excessive working hours in factories - 1-s2.0-S0160791X25001496-main.pdf](#)

Worker detection + excessive hours monitoring with YOLOv8l achieving 96.6% accuracy + blockchain smart contracts for immutable compliance records → exact POPW use case with payment automation.

10. Blockchain IoT Defect Inspection

[s41598-025-86777-3-1-compressed.pdf](#)

ResNet defect detection + blockchain data storage with immutable traceability → parallels WorkerNet for industrial quality assurance with cryptographic audit trails via blockchain.

11. Construction Worker Activity Analysis

[Construction Activity Analysis of Workers Based on Human Posture Estimation Information - 1-s2.0-S2095809923004691-main_compressed.pdf](#)

Human pose estimation for analyzing worker activities (taking, inspecting, checking) in construction = bottle checking 5-step workflow. Shows pose estimation directly enables activity verification.

12. Repetitive Assembly Action Recognition

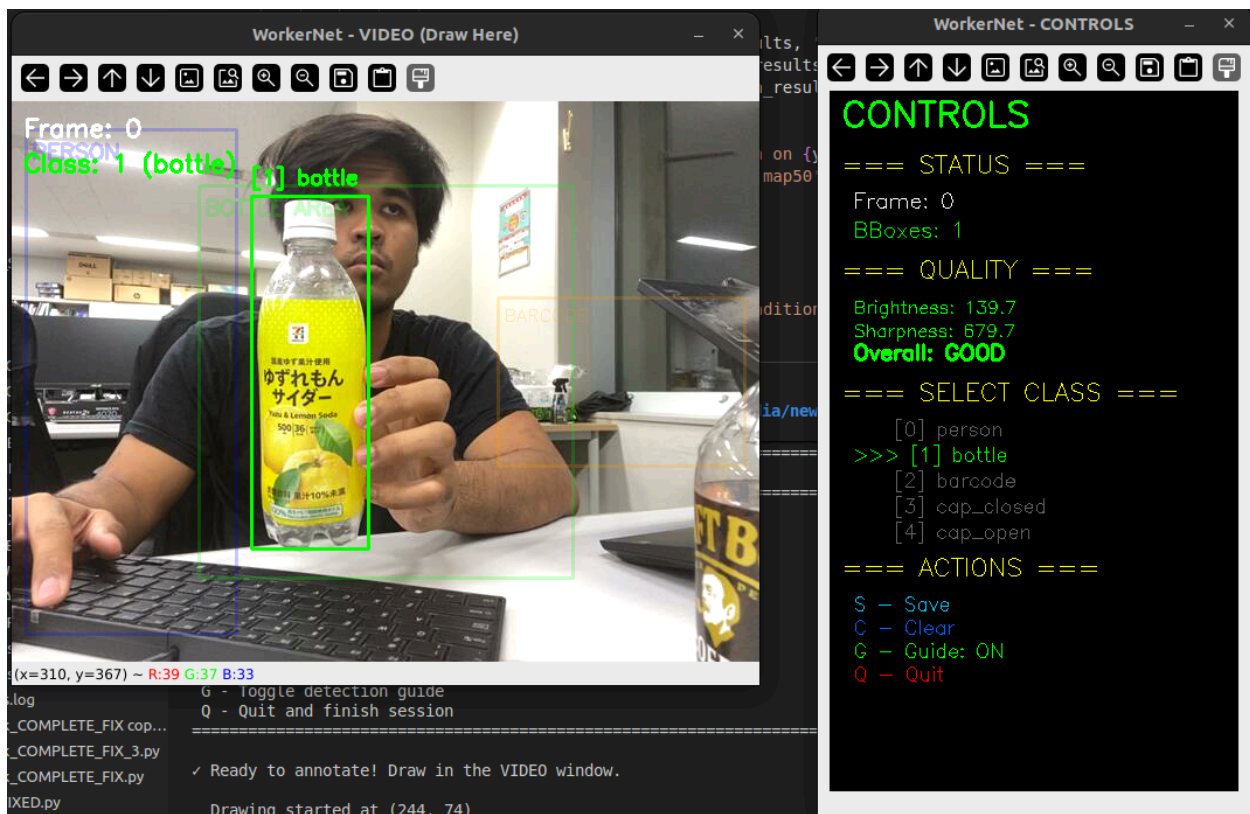
[Repetitive assembly action recognition based on object detection and pose estimation - 1-s2.0-S0278612520300625-main.pdf](#)

Combines object detection + pose estimation for recognizing assembly worker actions in manufacturing → the exact detection + pose combination but for general industrial activities not bottles.

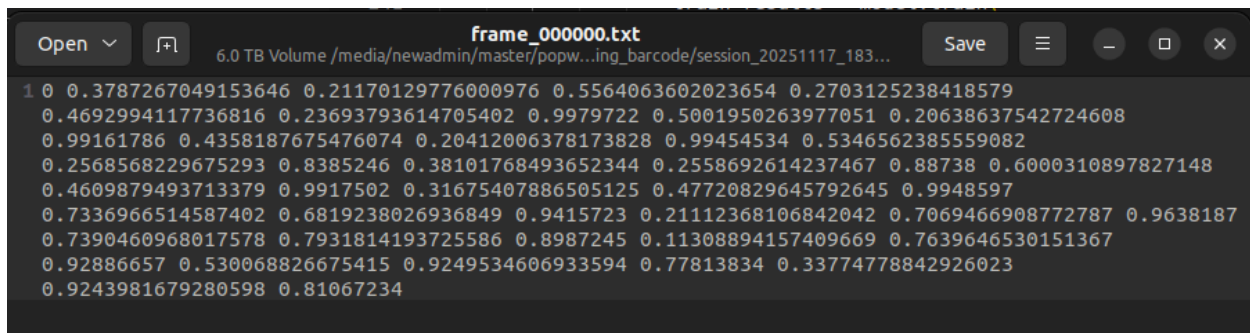
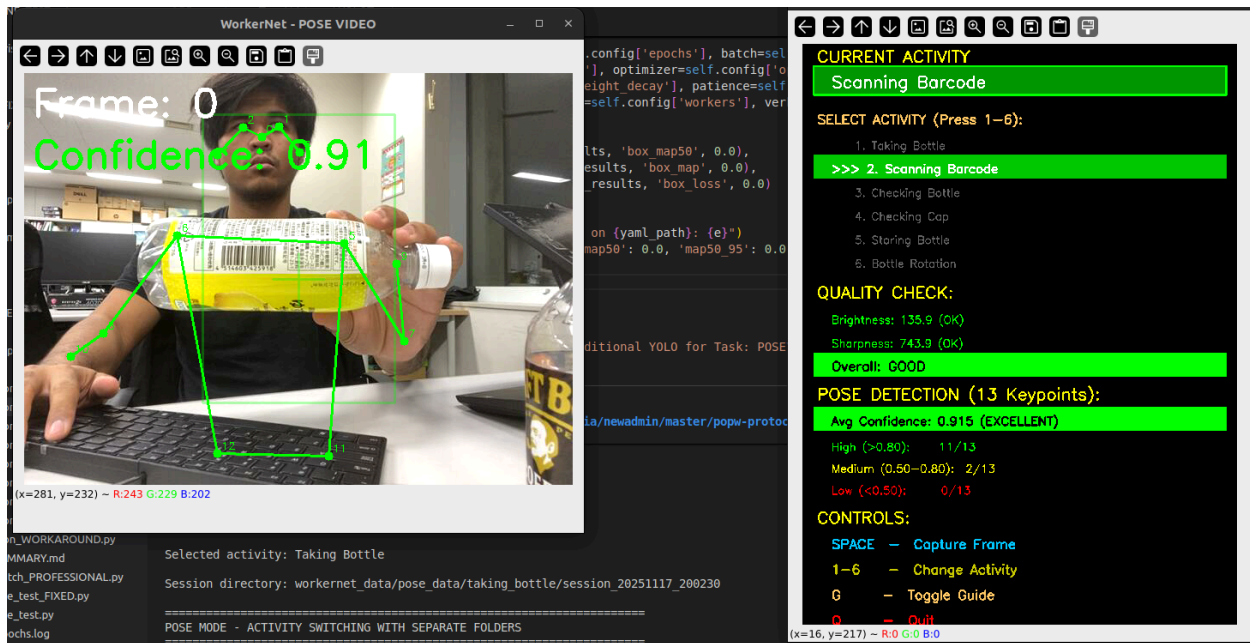
13. Unified Pedestrian Detection + Pose + Tracking

[Unified Deep Learning for Real-Time Pedestrian Detection, Pose Estimation, and Tracking - Paper_19-Unified_Deep_Learning_for_Real_Time_Pedestrian_Detection.pdf](#)

Real-time pedestrian detection + pose + tracking in single network—shows how unified frameworks handle temporal consistency better than sequential YOLO→OpenPose→tracker pipelines.



```
workernet_data > detection_data > session_20251117_184016 > labels > frame_000005.txt
1 1 0.169140625 0.4083333333333333 0.09140625 0.4388888888888889
2
```

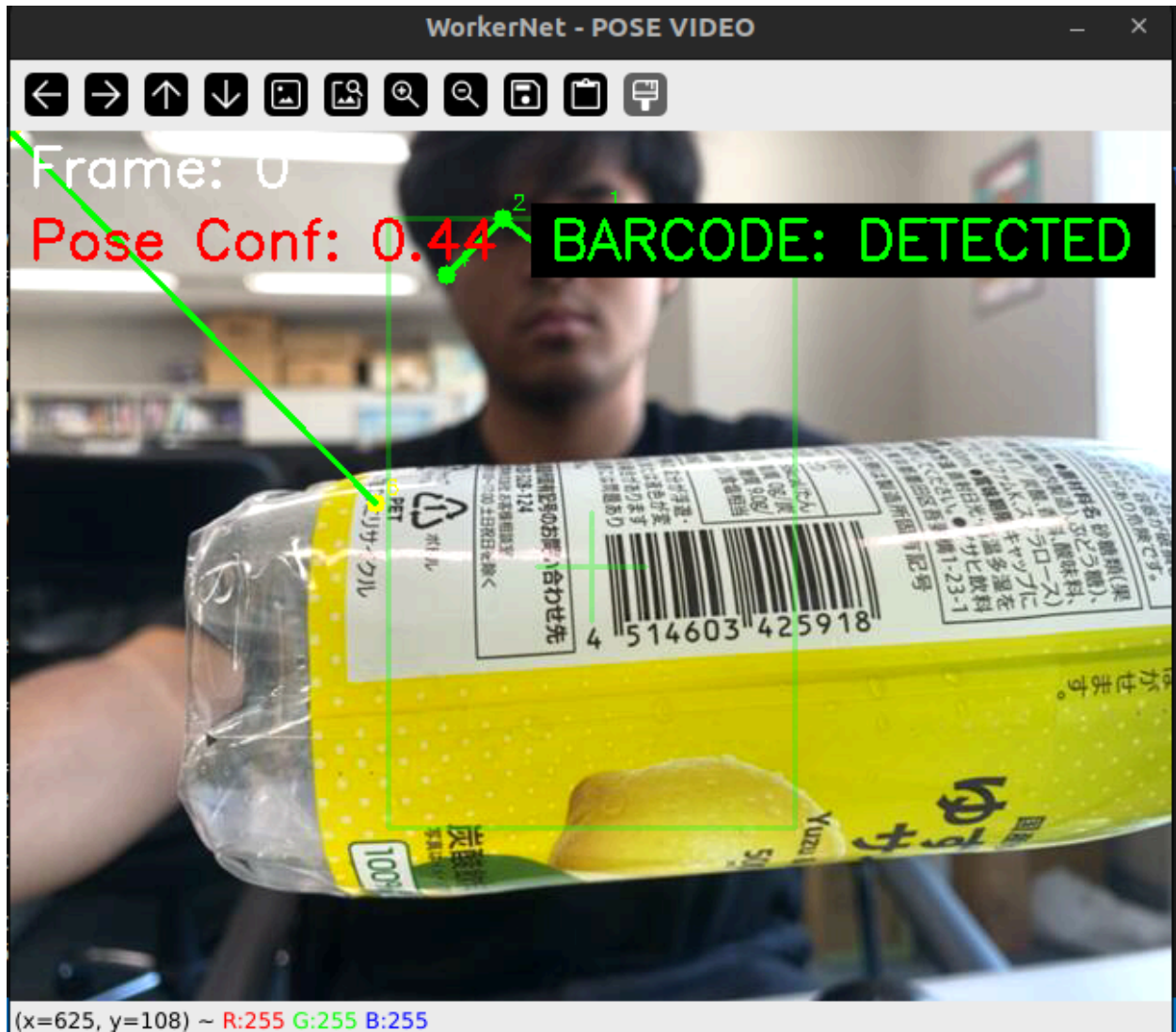


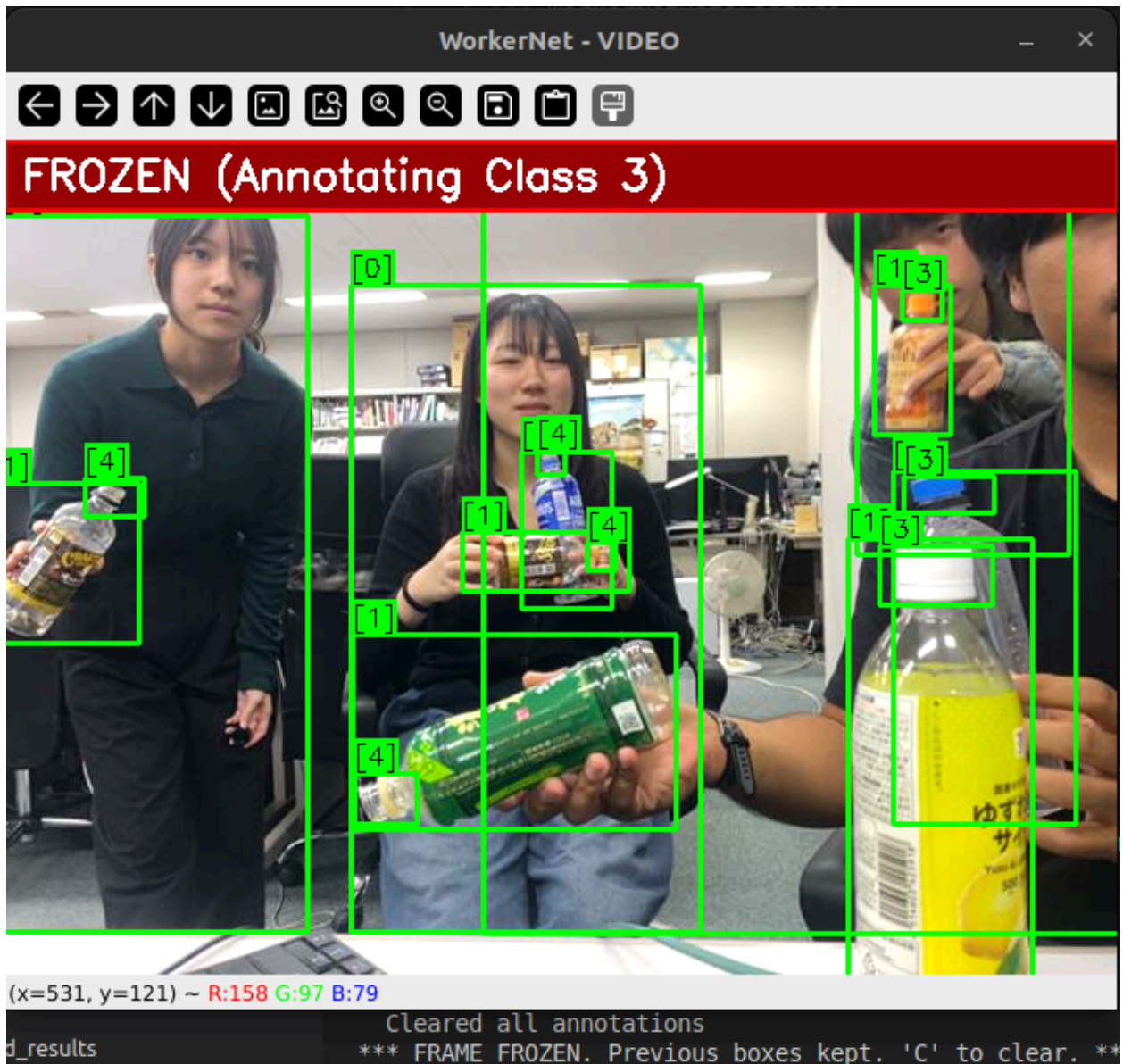
Initial plan was,

1. Taking the bottle
2. Scanning barcode
3. Checking bottle
4. Bottle Rotation
5. Storing Bottle

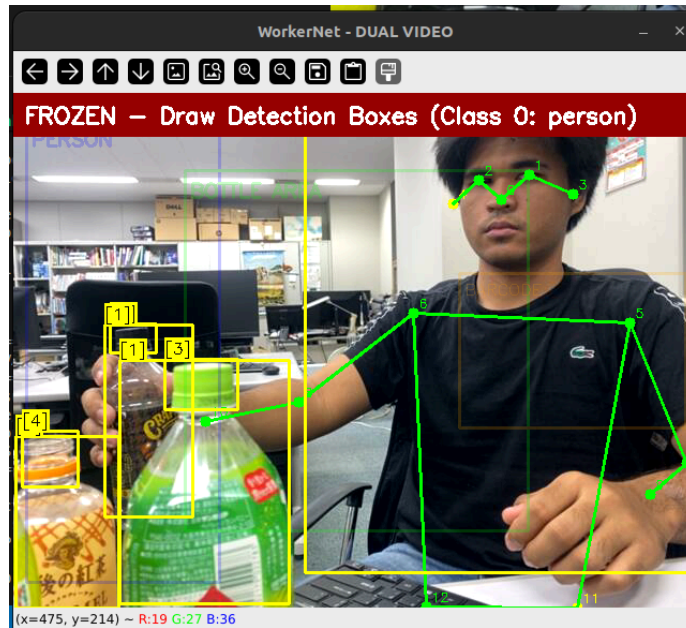
1. Scanning the barcode → stored to database

2. Begin pose (Checking, Rotation, Storing) → there will be UI to confirm those activity checked correctly)
3. do i need to make my own model to read the barcode or is it better to just use library? (because barcode is standarize)





```
workernet_data > detection_data > session_20251121_160830 > labels > Frame_000029.txt
1 0 0.0671875 0.3458333333333333 0.1375 0.5722222222222222
2 0 0.233984375 0.3736111111111111 0.15703125 0.5166666666666667
3 0 0.395703125 0.3368055555555556 0.36171875 0.5930555555555556
4 0 0.43046875 0.18125 0.0953125 0.2986111111111111
5 1 0.026171875 0.3375 0.06796875 0.1277777777777778
6 1 0.22890625 0.4722222222222222 0.1453125 0.1555555555555556
7 1 0.24296875 0.3361111111111111 0.075 0.0472222222222222
8 1 0.25234375 0.3111111111111111 0.040625 0.125
9 1 0.4203125 0.4944444444444444 0.0828125 0.3527777777777778
10 1 0.440234375 0.4048611111111111 0.08203125 0.2819444444444444
11 1 0.4078125 0.175 0.034375 0.1194444444444444
12 4 0.04921875 0.2847222222222222 0.0265625 0.0305555555555556
13 4 0.17109375 0.5256944444444444 0.028125 0.0402777777777778
14 4 0.26796875 0.3319444444444444 0.0125 0.0222222222222222
15 4 0.2453125 0.2569444444444444 0.0125 0.0194444444444444
16 3 0.418359375 0.3472222222222222 0.05078125 0.0472222222222222
17 3 0.42421875 0.2826388888888889 0.040625 0.0291666666666667
18 3 0.412109375 0.13125 0.01796875 0.0236111111111111
```

```
clean.py workernet_comparison_CORRECTED_1.py workernet_comparison_CORRECTED_1_copy.py capture_pose_data.py capture_pose_data_v2_updated.py frame_000000.txt x
data_collection > workernet_data > dual_data > taking_bottle > session_20251124_200322_pose > labels > frame_000000.txt
1 0 0.5851001 0.21161497 0.5194098 0.29479057 0.7678184509277344 0.2146804173787435 0.9969021 0.8031224250793457 0.17125247319539388 0.989405 0.73090043067
2
```

```
clean.py workernet_comparison_CORRECTED_1.py workernet_comparison_CORRECTED_1_copy.py capture_pose_data.py
data_collection > workernet_data > dual_data > taking_bottle > session_20251124_200322_detection > labels > frame_000000.txt
1 1 0.10859375 0.5 0.1125 0.3138888888888889
2 3 0.1078125 0.37222222222222223 0.05 0.05555555555555555
3 1 0.14765625 0.36388888888888889 0.053125 0.19166666666666666
4 4 0.1390625 0.2861111111111111 0.0234375 0.03055555555555555
5 0 0.337109375 0.3527777777777778 0.31640625 0.5861111111111111
6
```

```
Validating yolo validation runs/data_check4/weights/best.pt...
Ultralytics YOLOv8.2.103 Python-3.10.19 torch-2.9.1+cu128 CUDA:0 (NVIDIA GeForce RTX 3060, 12044MiB)
Model summary (fused): 168 layers, 3,006,623 parameters, 0 gradients, 8.1 GFLOPs
Class Images Instances Box(P R mAP50 mAP50-95): 100% 3/3 [00:00<00:00, 7.24it/s]
all 38 244 0.554 0.251 0.249 0.128
person 36 37 0.807 0.892 0.837 0.453
bottle 33 106 0.428 0.264 0.308 0.141
barcode 7 8 1 0 0.00404 0.00214
cap closed 26 63 0.404 0.0635 0.077 0.0363
cap open 17 30 0.132 0.0333 0.0164 0.00631
Speed: 0.4ms preprocess, 2.6ms inference, 0.0ms loss, 1.0ms postprocess per image
Results saved to yolo_validation_runs/data_check4

Validating...
Ultralytics YOLOv8.2.103 Python-3.10.19 torch-2.9.1+cu128 CUDA:0 (NVIDIA GeForce RTX 3060, 12044MiB)
Model summary (fused): 168 layers, 3,006,623 parameters, 0 gradients, 8.1 GFLOPs
val: Scanning /media/newadmin/master/popw-protocol/workernet/refined_workernet/workernet_data/detection data/session_20251121_160830/labels.cache... 38 images,
Class Images Instances Box(P R mAP50 mAP50-95): 100% 5/5 [00:00<00:00, 8.18it/s]
all 38 244 0.554 0.251 0.249 0.127
person 36 37 0.807 0.892 0.837 0.452
bottle 33 106 0.428 0.264 0.309 0.14
barcode 7 8 1 0 0.00409 0.00214
cap closed 26 63 0.403 0.0635 0.077 0.0364
cap open 17 30 0.132 0.0333 0.0164 0.0062
Speed: 1.1ms preprocess, 6.9ms inference, 0.0ms loss, 5.3ms postprocess per image
Results saved to yolo_validation_runs/data_check42
```

Short Term Planning (3)

Aa Name	📅 Date	☰ Tags
<u>Untitled</u>		
<u>Write paper and submission</u>	@December 31, 2025 → January 31, 2026	
<u>Address feedback, revise code and documentation</u>		
<u>Prepare figures, diagrams, and tables for paper</u>		
<u>POPW Implementation COMPLETE</u>	@December 15, 2025 → December 31, 2025	
<u>Address feedback, revise code and documentation</u>		
<u>Prepare figures, diagrams, and tables for paper</u>		
<u>Data and result collection for paper</u>	@December 8, 2025 → December 14, 2025	
<u>Address feedback, revise code and documentation</u>		
<u>Prepare figures, diagrams, and tables for paper</u>		
<u>Optimization</u>	@December 2, 2025 → December 14, 2025	
<u>Real world lab testing</u>	@November 25, 2025 → December 14, 2025	
<u>POPW protocol implementation</u>	@November 25, 2025 → December 14, 2025	
<u>Implement the spatiotemporal fingerprint</u>	@November 25, 2025 → December 14, 2025	
<u>Debugging</u>	@November 11, 2025 → December 14, 2025	
<u>Untitled</u>		
<u>Data Collection</u>	@November 11, 2025 → December 8, 2025	
<u>Finalize architecture and prepare result</u>	@October 31, 2025 → December 8, 2025	

Aa Name	📅 Date	☰ Tags
<u>Complete workernet training & benchmarking</u>	@October 14, 2025 → December 8, 2025	
<u>Untitled</u>	@November 17, 2023 → November 30, 2023	

Identify the base best model → Compare

Yolo for the first prototype → At least this model able to train and implement correctly